

1. 整体目标

计算： $C=A \times B$

其中 A、B、C 都是稀疏矩阵，A 是行稀疏矩阵，B 是列稀疏矩阵

对于一个非零元素：

$$A(i,k)$$

它需要和 B 的第 k 行所有非零元素相乘：

$$A(i,k) \times B(k,j)$$

所以一个 $A(i,k)$ 的真实任务量是：

$$nnz(B[k,:])$$

如果每个 PE 内有 N_MAC 个 MAC，那么这个任务的周期权重为：

$$cycle_work(A(i,k)) = \left\lceil \frac{nnz(B[k,:])}{N_MAC} \right\rceil$$

2. 硬件输入格式

A 矩阵输入：CSR 格式 A_row_ptr A_col_idx A_val

B 矩阵输入：CSC 格式 B_col_ptr B_row_idx B_val

3. 整体计算流程

输入 A CSR + B CSC



统计 B 每行非零数 b_row_nnz



构造 B 的 row-wise view



扫描 A CSR, 计算每一行 A 的任务量 row_cyc



计算 TotalCycleWork



使用“动态剩余目标 + 最近边界”切分 A 行



生成每个 PE 的 row_start / row_end



PE 阵列并行计算



PE 内部 local accumulator 聚合 partial products



输出 C CSR

5. 统计 B 每行非零数、构造 B 的 row-wise

先顺序扫描 B_row_idx, 输出 b_row_nnz[k] = B 第 k 行的非零元素数量

构造 B 的 csc 格式 B_row_ptr; B_row_col_idx ; B_row_val

6.计算 A 每行任务量

对于 A 第 i 行: A(i,k0), A(i,k1), A(i,k2), ...

$$row_cyc[i] = \sum_{A(i,k) \neq 0} \left\lceil \frac{b_row_nnz[k]}{N_MAC} \right\rceil$$

周期任务量:

7: 动态剩余目标 + 最近边界切分

为了尽量让每个 pe 分配的任务量均等，采用动态剩余目标 + 最近边界切分的方法。

为了简化聚合操作，减少硬件资源利用率，计划采用将 A 的同一行都分配给一个 pe 的分配策略。

如果直接在 A 的非零元素中间切分，可能会出现一个问题：A 的同一行 i 被分给多个 PE 例如：

PE0 计算 A 第 10 行的一部分

PE1 计算 A 第 10 行的另一部分

那么它们都会产生：C(10, j)，这样最后就需要跨 PE 合并同一行 C 的 partial sums，聚合会变复杂。所以建议将顺序切分改成按 A 的行顺序扫，按每一行的总 cycle_work 做切分，每个 PE 负责一段连续的 A 行，这样的好处是每一行 C 只由一个 PE 生成，因此聚合只需要在 PE 内部完成，不需要跨 PE 合并同一行 C。

动态剩余目标：每分配一个 PE，就重新计算剩余任务的平均值；

对于当前 PE：

$\text{remaining_work} = \text{TotalCycleWork} - \text{assigned_work}$

$\text{remaining_pe} = N_PE - \text{cur_pe}$

这样前面 PE 如果分多了，后面 PE 的目标会自动降低；

前面 PE 如果分少了，后面 PE 的目标会自动升高。

最近边界切分

假设当前 PE 已经累积：cur_load；当前行任务量：w = row_cyc[i]；加入当前行后：

$\text{new_load} = \text{cur_load} + w$

当 new_load 达到或超过 dynamic_target 时，比较两个误差：

$\text{err_before} = \text{dynamic_target} - \text{cur_load}$

$\text{err_after} = \text{new_load} - \text{dynamic_target}$

如果 $\text{err_after} \leq \text{err_before}$ ，说明把当前行给当前 PE 更接近目标。否则当前行留给下一个 PE

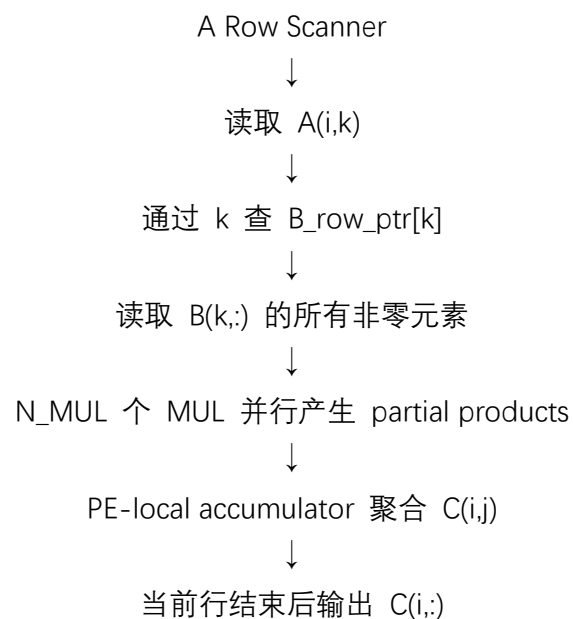
8: 创建 PE Task Table

调度完成后，每个 PE 不需要保存大量 task，只需要保存一段 A 矩阵的连续行范围：

```
PE_task_desc[pe_id] = {  
    valid,  
    row_start,  
    row_end,  
    A_ptr_start,  
    A_ptr_end,  
    estimated_cycle  
}
```

这样的话，PE 启动后只需要顺序扫描自己负责的 A 的元素。

9: PE 内部计算流程



因为按 A 行切分，且每个 PE 负责一段连续 A 行，所以每一行 C 只由一个 PE 生成，因此不需要跨 PE 合并同一行 C。这是本方案重要的优势。

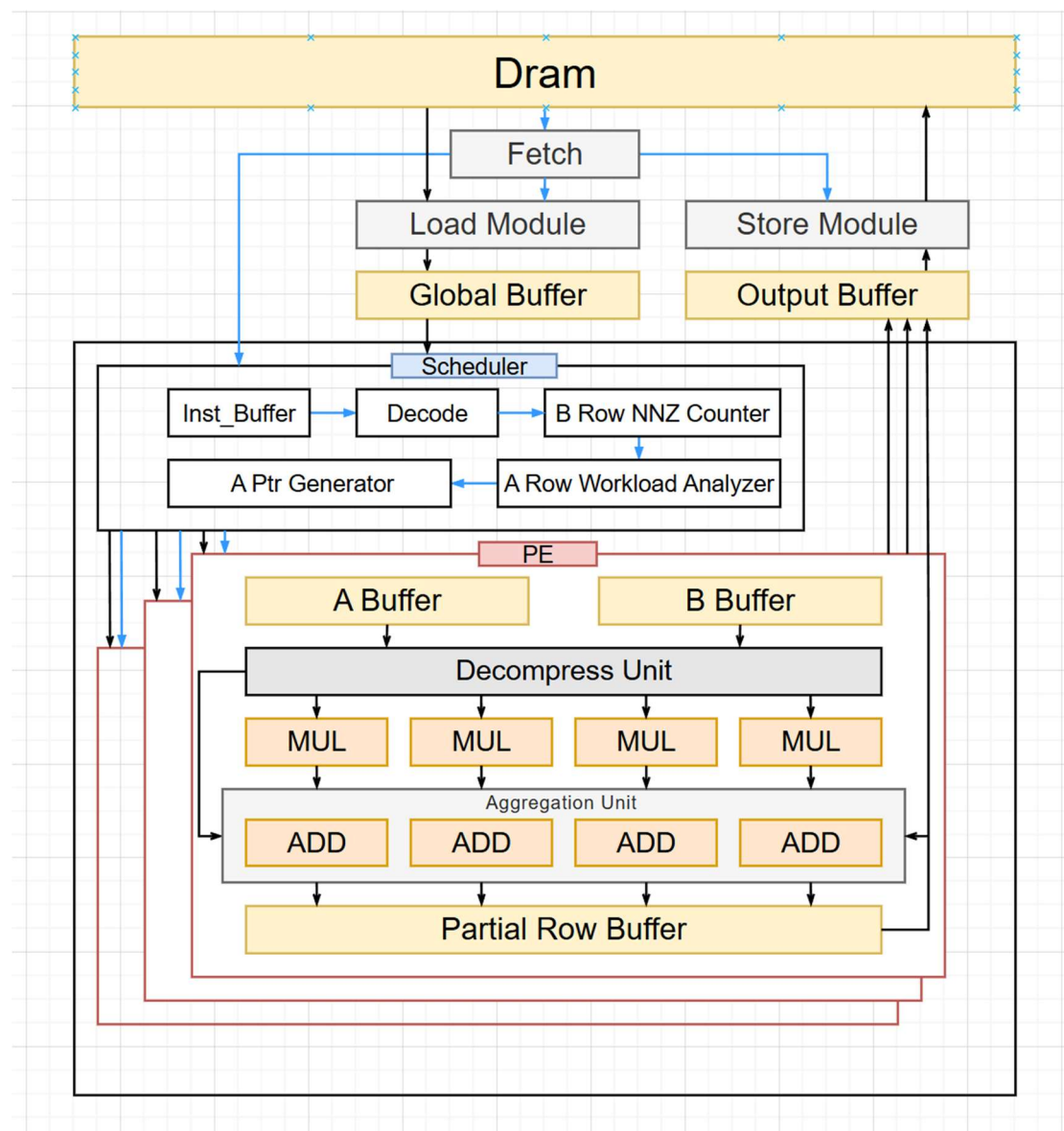
10: CSR 输出

分为以下过程：

1: 输出每行 C 的 row_nnz 和 row data

2: 所有 PE 完成后, 对 C_row_nnz 做前缀和, 生成 C_row_ptr

11: 整体硬件规划



12： 软仿结果

Case	A	B	B 稀疏度	Element_split	Row_split	+周期
A_0×B_0	251 × 257	257 × 121	15.18%	92.23%	90.90%	+1.5%
A_1×B_1	32 × 317	317 × 6	13.62%	32.61%	27.57%	+18.3%
A_2×B_2	335 × 335	335 × 135	14.78%,	92.57%	91.40%	+1.3%